Actual HW3

Sam Olson

Q1

Using the mixture model (1) and (2) fit the data in Data Set 1 and Data Set 2 separately using maximum likelihood. Produce approximate 95% confidence intervals for the modes of the distributions – these are the value of the parameter ξ in (2). Conduct any additional analysis needed to construct a likelihood ratio test of a reduced model in which the parameters ξ and θ are identical for both data sets against a full model in which those parameters differ between the two heights. Graph the extreme value density or densities for whichever model (full or reduced) you select.

Data model:

$$f(y \mid \mu_i) = \frac{1}{(2\pi)^{1/2}} \exp\left[-\frac{1}{2}(y-\mu_i)^2\right]. \tag{1}$$

Mixing distribution:

$$g(\mu \mid \xi, \theta) = \frac{1}{\theta} \exp\left(-\frac{\mu-\xi}{\theta}\right) \exp\left[-\exp\left(-\frac{\mu-\xi}{\theta}\right)\right], \qquad \theta > 0. \tag{2}$$

A:

For each height (50m and 80m), Normal–Extreme Value mixture models were fit by maximum likelihood. Because the latent daily mean μ is unobserved, the likelihood for an observed wind speed y is defined through the marginal mixture density

$$m(y \mid \xi, \theta) = \int f(y \mid \mu)\, g(\mu \mid \xi, \theta)\, d\mu, \qquad \ell(\xi, \theta) = \sum_{i=1}^{n} \log m(y_i \mid \xi, \theta),$$

where f is the Normal density in (1) and g is the Extreme Value density in (2). Each $\ell(\xi, \theta)$ was maximized separately for each data set to obtain MLEs $(\hat{\xi}_{50}, \hat{\theta}_{50})$ and $(\hat{\xi}_{80}, \hat{\theta}_{80})$.

This integral prevents the model from reducing to a standard regression formulation, as the latent μ_i are not directly estimated but integrated out.

Then, approximate 95% confidence intervals were constructed for the mode parameter ξ (and for θ) using the Wald approximation at the MLE. To validate these intervals, profile likelihood intervals were also computed: For fixed ξ (resp. θ), θ (resp. ξ) was re-maximized over the remaining parameter and the likelihood-ratio cutoff

$$2\{\ell(\hat{\xi}, \hat{\theta}) - \ell(\xi, \hat{\theta}(\xi))\} \leq \chi^2_{1,0.95},$$

The profile intervals provide a likelihood-based check on the Wald approximation.

Direct Likelihood Evaluation

Table 1: MLEs and Wald 95% confidence intervals for ξ.

Height	$\hat{\xi}$	Lower 95%	Upper 95%
50m	4.392	3.719	5.066
80m	6.717	5.646	7.787

Table 2: MLEs and Wald 95% confidence intervals for θ.

Height	$\hat{\theta}$	Lower 95%	Upper 95%
50m	2.052	1.577	2.671
80m	3.522	2.771	4.477

Profile Likelihood Evaluation

Table 3: Profile likelihood confidence intervals for ξ.

Height	95% Profile Likelihood CI for ξ
50m	[3.700, 5.075]
80m	[5.633, 7.823]

Table 4: Profile likelihood confidence intervals for θ.

Height	95% Profile Likelihood CI for θ
50m	[1.582, 2.686]
80m	[2.793, 4.520]

Likelihood Ratio Test

A likelihood ratio test was constructed comparing a reduced model, in which a common (ξ, θ) was estimated (the reduced-model estimates are omitted, as the full model was ultimately selected) from the pooled data across both heights, to a full model with separate (ξ_{50}, θ_{50}) and (ξ_{80}, θ_{80}).

The test statistic

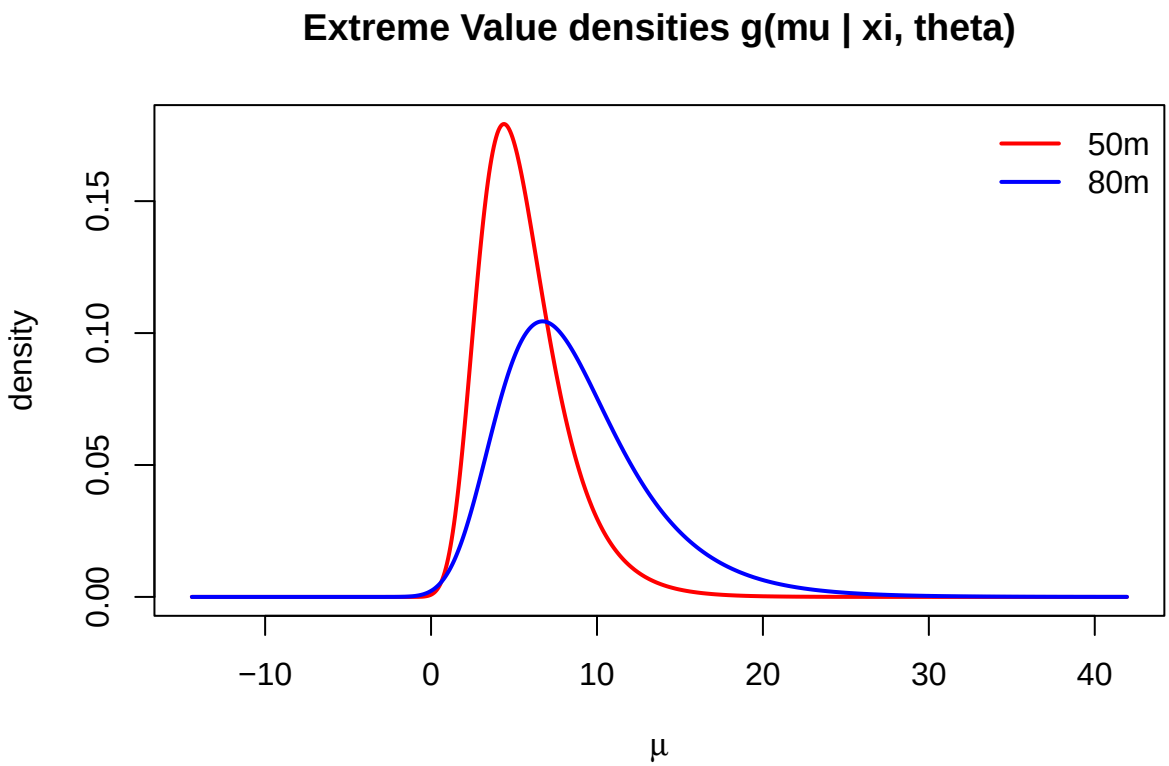
$$\mathrm{LR} = 2\{\ell_{\text{full}} - \ell_{\text{reduced}}\}$$

was compared to a χ^2 distribution with 2 degrees of freedom. The results provided strong evidence against the reduced model, indicating that a common mixing distribution does not adequately describe both heights.

Based on this conclusion, the fitted Extreme Value densities $g(\mu \mid \xi, \theta)$ were then plotted under the selected (full) model using the corresponding maximum likelihood estimates, enabling direct comparison of the latent wind-speed distributions across heights.

Table 5: Likelihood ratio test for (ξ, θ) (Full vs. Reduced model).

Hypothesis	LR Statistic	df	p-value
H_0: common (ξ, θ) across heights	18.579	2	9.24e-05



Q2

In (2), $E(\mu) \approx \xi + 0.577\theta$, where 0.577 is Euler's constant (don't confuse with Euler's number). Using this expression, produce maximum likelihood estimate or estimates and 95% confidence interval or confidence intervals for the expected true maximum wind speed from your final model. Note that this will also be the expected observed maximum wind speed.

A:

Table 6: MLE and Wald 95% confidence intervals for expected true maximum wind speed.

Height	$\widehat{E(\mu)}$	SE	Lower 95%	Upper 95%
50m	5.577	0.396	4.802	6.353
80m	8.750	0.651	7.474	10.026

Q3

Find a maximum likelihood estimate or estimates (depending on your final model) of the median daily maximum wind speed or speeds. Compute the associated 95% approximate confidence interval or intervals.

A:

The median of Y under the fitted mixture model is defined as the (implicit) solution m to

$$F_Y(m \mid \xi, \theta) = 0.5,$$

where F_Y is the marginal CDF obtained by integrating out the latent μ.

Using the MLEs $(\hat{\xi}, \hat{\theta})$ from Q1, we then solve for

$$F_Y(m \mid \hat{\xi}, \hat{\theta}) = 0.5$$

numerically via root-finding to obtain the estimated median for each height.

Subsequent approximate 95% confidence intervals were then constructed via the delta method, treating the median as a function of $(\xi, \log\theta)$ and using the observed-information covariance from the MLE.

Table 7: MLE and Wald 95% confidence intervals for median daily maximum wind speed.

Height	$\widehat{\text{Median}(Y)}$	SE	Lower 95%	Upper 95%
50m	5.208	0.367	4.488	5.928
80m	8.049	0.601	6.871	9.227

Q4

Find a maximum likelihood estimate or estimates (depending on your final model) for the probability that true daily maximum wind speed is between 4 and 17 m/s. Compute 95% approximate confidence intervals for the estimate or estimates.

A

Under the fitted model, the probability that the latent true wind speed μ lies in $[4, 17]$ is

$$Pr(4 \leq \mu \leq 17) = G(17 \mid \xi, \theta) - G(4 \mid \xi, \theta),$$

where G is the Extreme Value CDF. Using the MLEs $(\hat{\xi}, \hat{\theta})$ from Q1. This expression is evaluated directly to obtain the estimate for each height.

For context, this corresponds to the model-based analogue of an ECDF estimate, where instead of using the sample ECDF of μ (which is unobserved), we use the fitted parametric CDF $G(\cdot \mid \hat{\xi}, \hat{\theta})$.

Then, the subsequent approximate 95% confidence intervals were then constructed via the delta method, treating the probability as a function of $(\xi, \log \theta)$ and using the observed-information covariance from the MLE.

Table 8: MLE and 95% Wald confidence intervals for estimated probability $Pr(4 \leq \mu \leq 17)$.

Height	$\widehat{Pr(4 \leq \mu \leq 17)}$	SE	Lower 95%	Upper 95%
50m	0.700	0.060	0.583	0.817
80m	0.832	0.046	0.743	0.922

Q5

What would you conclude about the question of whether there is or is not a benefit to placing turbine hubs at either 50 or 80 m in height?

A:

Overall, there is benefit to placing turbine hubs at 80m.

In Q1, the likelihood ratio test provided strong evidence that the wind speed distributions at 50m and 80m differ ($p \approx 9.24 \times 10^{-5}$), leading us to adopt the full model with separate parameter estimates for each height. The fitted extreme value mixing distributions therefore represent statistically distinct underlying wind distributions.

Under the full model, all key summaries indicate systematically higher wind speeds at 80m. The estimated expected true daily maximum wind speed is approximately 3.2 m/s higher at 80m (8.75 vs. 5.58 m/s), and the estimated median differs by a similar margin (8.05 vs. 5.21 m/s). Furthermore, the estimated probability that true daily maximum wind speed falls within the range of 4–17 m/s is substantially larger at 80m (0.832 vs. 0.700), with confidence intervals suggesting that this difference is unlikely to be explained by sampling variability alone (very minor overlap of probabilities in Q4).

Taken together, a turbine hub height of 80m is associated with more favorable wind speed characteristics, though the above conclusions are conditional on the correctness of the model assumptions and on future wind patterns being similar to those represented in the data.

Q6

As an exercise in learning about statistical procedures, your friend would like to see a Monte Carlo approximation to the expected true wind speed for the two heights separately. Produce such approximations along with 95% Monte Carlo intervals. Your friend understands intuitively that these approximations and the maximum likelihood estimates of Question 2 should have the same or nearly the same values, but marvels at the fact that the intervals associated with these approximations are so much more narrow than those computed using the asymptotic theory of likelihood estimation in Question 2. Explain why this is the case and what it means.

A:

Monte Carlo

Table 9: Monte Carlo approximation of $E(\mu)$ with 95% Monte Carlo intervals.

Height	M	$\widehat{E(\mu)}_{\text{MC}}$	Lower 95%	Upper 95%
50m	20000	5.583	5.546	5.620
80m	20000	8.706	8.644	8.768

Comparison - Concise

The Monte Carlo (MC) intervals are much narrower because they measure the simulation (numerical) error conditional on the estimated parameters $(\hat{\xi}, \hat{\theta})$. By contrast, asymptotic confidence intervals like those in Question 2 account for statistical uncertainty in the joint estimation of (ξ, θ) from finite data.

Comparison - More Details

For each height, we simulated

$$\mu^{(1)}, \ldots, \mu^{(M)} \sim \text{EV}(\hat{\xi}, \hat{\theta})$$

and estimated the expected true wind speed by

$$\widehat{E}_{MC}(\mu) = \bar{\mu}_M = \frac{1}{M} \sum_{m=1}^{M} \mu^{(m)}.$$

By the Central Limit Theorem for simulation,

$$\bar{\mu}_M \approx \mathcal{N}\left(E(\mu), \frac{\text{Var}(\mu)}{M}\right),$$

so an approximate 95% Monte Carlo interval is

$$\bar{\mu}_M \pm 1.96 \frac{s_M}{\sqrt{M}}.$$

The crucial point is that this interval treats $(\hat{\xi}, \hat{\theta})$ as fixed and quantifies only Monte Carlo approximation error, which decreases at rate $1/\sqrt{M}$ and can be made arbitrarily small by increasing M.

In contrast, the confidence interval in Question 2 is based on asymptotic likelihood theory for

$$E(\mu) = \xi + \gamma\theta, \qquad \gamma \approx 0.577,$$

and reflects uncertainty in estimating (ξ, θ) from $n = 50$ observations. That parameter uncertainty propagates into uncertainty about $E(\mu)$.

Thus, the Monte Carlo intervals are narrow because they measure numerical precision and not inferential uncertainty. The wider intervals from Question 2 are the appropriate measures of statistical uncertainty about the expected wind speed. To obtain a simulation-based interval comparable to Question 2, one would need to incorporate parameter uncertainty in some manner, rather than conditioning on fixed MLEs.

Appendix

```r
# ----------------------------
# Setup
# ----------------------------
library(tidyverse)
library(numDeriv)
library(here)

source(here("newtraph.txt"))
source(here("bisect.txt"))

m50 <- read.table(here("winddat1.txt"), header = TRUE)
m80 <- read.table(here("winddat2.txt"), header = TRUE)

y50 <- as.numeric(m50$y)
y80 <- as.numeric(m80$y)

# Euler constant
gamma <- 0.5772156649015329

# ============================================================
# Core model pieces (used throughout)
# ============================================================

g_mu <- function(mu, xi, theta) {
  # input:  mu, xi, theta
  # output: g(mu | xi, theta) (right-skew Gumbel / EV Type I density)
  z <- (mu - xi) / theta
  (1 / theta) * exp(-z - exp(-z))
}

m_y_stable <- function(y, xi, theta, zL = -10, zU = 30) {
  # input:  y, xi, theta; z-bounds for stable integration
  # output: m(y | xi, theta) = int f(y|mu) g(mu|xi,theta) dmu
  #         computed via z = (mu - xi)/theta:  m(y) = int phi(y | xi + theta*z, 1) * exp(-z - exp(-z))

  integrand <- function(z) {
    log_int <- dnorm(y, mean = xi + theta * z, sd = 1, log = TRUE) - z - exp(-z)
    exp(log_int)
  }

  val <- integrate(
    integrand,
    lower = zL,
    upper = zU,
    rel.tol = 1e-8,
    abs.tol = 0,
    subdivisions = 5000
  )$value

  pmax(val, .Machine$double.xmin)
}
```

```r
log_m_vec_stable <- function(y, xi, theta, zL = -10, zU = 30) {
  # input:  y vector, xi, theta
  # output: vector of log m(y_i | xi, theta)
  vapply(y, function(yy) log(m_y_stable(yy, xi, theta, zL, zU)), numeric(1))
}

logLik_one <- function(par, y, zL = -10, zU = 30) {
  # input:  par = (xi, logtheta), data y
  # output: log-likelihood ell(xi,theta)
  xi <- par[1]
  theta <- exp(par[2])
  sum(log_m_vec_stable(y, xi, theta, zL = zL, zU = zU))
}

nll_one <- function(par, y, zL = -10, zU = 30) {
  # input:  par = (xi, logtheta), data y
  # output: negative log-likelihood
  -logLik_one(par, y = y, zL = zL, zU = zU)
}

ev_start <- function(y) {
  # input:  y vector
  # output: start = (xi0, logtheta0) from rough moment matching
  v <- var(y)
  theta0 <- sqrt(pmax((v - 1) * 6 / pi^2, 1e-6))
  xi0 <- mean(y) - gamma * theta0
  c(xi0, log(theta0))
}

make_ders_one <- function(zL = -10, zU = 30) {
  # input:  z bounds
  # output: ders(par, dat) -> list(logLik, grad, Hessian)
  function(par, dat) {
    f <- function(pp) logLik_one(pp, y = dat, zL = zL, zU = zU)
    ll <- f(par)
    g <- grad(f, par)
    H <- hessian(f, par)
    list(ll, g, H)
  }
}

fit_one_newtraph <- function(y, zL = -10, zU = 30) {
  # input:  y vector
  # output: MLEs + observed-information covariance on (xi, logtheta) scale
  start <- ev_start(y)
  ders <- make_ders_one(zL = zL, zU = zU)

  out <- newtraph(
    ders, dat = y, x0 = start,
    quiet = TRUE, print_final = FALSE,
    step_scale = 0.5, max_halving = 60
  )
```

```r
  par_hat <- as.numeric(out[[1]])
  xi_hat <- par_hat[1]
  theta_hat <- exp(par_hat[2])

  V <- out[[3]]  # (-H)^{-1} at solution on (xi, logtheta)
  se_xi <- sqrt(V[1, 1])

  list(
    xi_hat = xi_hat,
    theta_hat = theta_hat,
    logLik = out[[2]],
    fInf = V,
    se_xi = se_xi,
    ci_xi_wald = xi_hat + c(-1, 1) * 1.96 * se_xi,
    start = start,
    z_bounds = c(zL, zU)
  )
}

wald_theta_from_fit <- function(fit, alpha = 0.05) {
  # input:  fit from fit_one_newtraph()
  # output: Wald CI for theta using delta method from logtheta
  z <- qnorm(1 - alpha / 2)

  se_logtheta <- sqrt(fit$fInf[2, 2])
  logtheta_hat <- log(fit$theta_hat)

  ci_logtheta <- logtheta_hat + c(-1, 1) * z * se_logtheta
  se_theta <- fit$theta_hat * se_logtheta

  list(
    theta_hat = fit$theta_hat,
    se_logtheta = se_logtheta,
    se_theta = se_theta,
    ci_theta = exp(ci_logtheta)
  )
}


# ============================================================
# Q1: Separate fits (50m and 80m) + Wald CIs
# ============================================================

fit50 <- fit_one_newtraph(y50, zL = -100, zU = 100)
fit80 <- fit_one_newtraph(y80, zL = -100, zU = 100)

th50_wald <- wald_theta_from_fit(fit50)
th80_wald <- wald_theta_from_fit(fit80)


# ============================================================
# Q1: Profile likelihood CIs for xi and theta (via optimize + uniroot)
# ============================================================
```

```r
profile_logLik_xi <- function(xi_fixed, y, logtheta_center, zL = -10, zU = 30) {
  # input:  fixed xi, data y
  # output: profile logLik ell_prof(xi) (re-max over logtheta)
  obj <- function(logtheta) nll_one(c(xi_fixed, logtheta), y = y, zL = zL, zU = zU)

  opt <- optimize(
    obj,
    interval = c(logtheta_center - log(20), logtheta_center + log(20))
  )

  -opt$objective
}

ci_xi_profile <- function(y, xi_hat, theta_hat, logLik_hat, zL = -10, zU = 30, alpha = 0.05) {
  # input:  data y and MLEs
  # output: LR-based profile CI for xi
  cutoff <- qchisq(1 - alpha, df = 1)

  target <- function(xi) {
    ell_prof <- profile_logLik_xi(xi_fixed = xi, y = y, logtheta_center = log(theta_hat), zL = zL, zU =
    2 * (logLik_hat - ell_prof) - cutoff
  }

  step0 <- max(0.25, 0.5 * theta_hat)

  left <- xi_hat - step0
  step <- step0
  for (k in 1:80) {
    tl <- target(left)
    if (is.finite(tl) && tl < 0) { left <- left - step; step <- step * 1.2 } else break
  }

  right <- xi_hat + step0
  step <- step0
  for (k in 1:80) {
    tr <- target(right)
    if (is.finite(tr) && tr < 0) { right <- right + step; step <- step * 1.2 } else break
  }

  lower <- uniroot(target, interval = c(left, xi_hat))$root
  upper <- uniroot(target, interval = c(xi_hat, right))$root

  c(lower, upper)
}

profile_logLik_theta <- function(logtheta_fixed, y, xi_center, zL = -10, zU = 30) {
  # input:  fixed logtheta, data y
  # output: profile logLik ell_prof(logtheta) (re-max over xi)
  obj <- function(xi) nll_one(c(xi, logtheta_fixed), y = y, zL = zL, zU = zU)

  theta_fixed <- exp(logtheta_fixed)

  opt <- optimize(
```

```r
    obj,
    interval = c(xi_center - 20 * theta_fixed, xi_center + 20 * theta_fixed)
  )

  -opt$objective
}

ci_theta_profile <- function(y, xi_hat, theta_hat, logLik_hat, zL = -10, zU = 30, alpha = 0.05) {
  # input:  data y and MLEs
  # output: LR-based profile CI for theta
  cutoff <- qchisq(1 - alpha, df = 1)

  target <- function(logtheta) {
    ell_prof <- profile_logLik_theta(logtheta_fixed = logtheta, y = y, xi_center = xi_hat, zL = zL, zU =
    2 * (logLik_hat - ell_prof) - cutoff
  }

  lt_hat <- log(theta_hat)
  step0 <- log(1.5)

  left <- lt_hat - step0
  step <- step0
  for (k in 1:80) {
    tl <- target(left)
    if (is.finite(tl) && tl < 0) { left <- left - step; step <- step * 1.2 } else break
  }

  right <- lt_hat + step0
  step <- step0
  for (k in 1:80) {
    tr <- target(right)
    if (is.finite(tr) && tr < 0) { right <- right + step; step <- step * 1.2 } else break
  }

  lower_lt <- uniroot(target, interval = c(left, lt_hat))$root
  upper_lt <- uniroot(target, interval = c(lt_hat, right))$root

  exp(c(lower_lt, upper_lt))
}

ci50_xi_prof <- ci_xi_profile(y50, fit50$xi_hat, fit50$theta_hat, fit50$logLik, zL = -100, zU = 100)
ci80_xi_prof <- ci_xi_profile(y80, fit80$xi_hat, fit80$theta_hat, fit80$logLik, zL = -100, zU = 100)

ci50_th_prof <- ci_theta_profile(y50, fit50$xi_hat, fit50$theta_hat, fit50$logLik, zL = -100, zU = 100)
ci80_th_prof <- ci_theta_profile(y80, fit80$xi_hat, fit80$theta_hat, fit80$logLik, zL = -100, zU = 100)


# ============================================================
# Q1: Likelihood ratio test (re-fit reduced + full without optim())
# Uses newtraph() with numeric grad/Hessian on the joint parameter vector.
# ============================================================

logLik_full <- function(par, y50, y80, zL = -100, zU = 100) {
```

```r
  # input:  par = (xi50, logth50, xi80, logth80), data y50,y80
  # output: ell_full = ell50(xi50,th50) + ell80(xi80,th80)
  xi50 <- par[1]; th50 <- exp(par[2])
  xi80 <- par[3]; th80 <- exp(par[4])

  sum(log_m_vec_stable(y50, xi50, th50, zL = zL, zU = zU)) +
    sum(log_m_vec_stable(y80, xi80, th80, zL = zL, zU = zU))
}

logLik_reduced <- function(par, y50, y80, zL = -100, zU = 100) {
  # input:  par = (xi, logtheta), pooled data y50,y80
  # output: ell_reduced = ell50(xi,th) + ell80(xi,th)
  xi <- par[1]; th <- exp(par[2])

  sum(log_m_vec_stable(y50, xi, th, zL = zL, zU = zU)) +
    sum(log_m_vec_stable(y80, xi, th, zL = zL, zU = zU))
}

make_ders_full <- function(zL = -100, zU = 100) {
  # input:  z bounds
  # output: ders(par, dat) -> list(logLik, grad, Hessian) for full model
  function(par, dat) {
    y50 <- dat$y50
    y80 <- dat$y80
    f <- function(pp) logLik_full(pp, y50 = y50, y80 = y80, zL = zL, zU = zU)
    ll <- f(par)
    g <- grad(f, par)
    H <- hessian(f, par)
    list(ll, g, H)
  }
}

make_ders_reduced <- function(zL = -100, zU = 100) {
  # input:  z bounds
  # output: ders(par, dat) -> list(logLik, grad, Hessian) for reduced model
  function(par, dat) {
    y50 <- dat$y50
    y80 <- dat$y80
    f <- function(pp) logLik_reduced(pp, y50 = y50, y80 = y80, zL = zL, zU = zU)
    ll <- f(par)
    g <- grad(f, par)
    H <- hessian(f, par)
    list(ll, g, H)
  }
}

fit_full_newtraph <- function(y50, y80, start, zL = -100, zU = 100) {
  # input:  start = (xi50, logth50, xi80, logth80)
  # output: full-model MLE + logLik
  ders <- make_ders_full(zL = zL, zU = zU)

  out <- newtraph(
    ders, dat = list(y50 = y50, y80 = y80), x0 = start,
```

```r
    quiet = TRUE, print_final = FALSE,
    step_scale = 0.5, max_halving = 60
  )

  par_hat <- as.numeric(out[[1]])
  list(
    xi50_hat = par_hat[1], th50_hat = exp(par_hat[2]),
    xi80_hat = par_hat[3], th80_hat = exp(par_hat[4]),
    logLik = out[[2]],
    fInf = out[[3]],
    start = start,
    out = out
  )
}

fit_reduced_newtraph <- function(y50, y80, start, zL = -100, zU = 100) {
  # input:  start = (xi, logtheta)
  # output: reduced-model MLE + logLik
  ders <- make_ders_reduced(zL = zL, zU = zU)

  out <- newtraph(
    ders, dat = list(y50 = y50, y80 = y80), x0 = start,
    quiet = TRUE, print_final = FALSE,
    step_scale = 0.5, max_halving = 60
  )

  par_hat <- as.numeric(out[[1]])
  list(
    xi_hat = par_hat[1], th_hat = exp(par_hat[2]),
    logLik = out[[2]],
    fInf = out[[3]],
    start = start,
    out = out
  )
}

# starts for constrained/unconstrained fits
start_full <- c(fit50$xi_hat, log(fit50$theta_hat), fit80$xi_hat, log(fit80$theta_hat))
start_red  <- ev_start(c(y50, y80))

full_hat <- fit_full_newtraph(y50, y80, start = start_full, zL = -100, zU = 100)
red_hat  <- fit_reduced_newtraph(y50, y80, start = start_red,  zL = -100, zU = 100)

LR <- 2 * (full_hat$logLik - red_hat$logLik)
pval <- 1 - pchisq(LR, df = 2)

# EV density plot under selected model (choose full if p < 0.05)
use_full <- (pval < 0.05)

if (use_full) {
  mu_grid <- seq(
    min(full_hat$xi50_hat - 6 * full_hat$th50_hat, full_hat$xi80_hat - 6 * full_hat$th80_hat),
    max(full_hat$xi50_hat + 10 * full_hat$th50_hat, full_hat$xi80_hat + 10 * full_hat$th80_hat),
```

```r
    length.out = 800
  )

  plot(mu_grid, g_mu(mu_grid, full_hat$xi50_hat, full_hat$th50_hat),
       type = "l", lwd = 2, col = "red",
       xlab = expression(mu), ylab = "density",
       main = "Extreme Value mixing densities g(mu | xi, theta)")

  lines(mu_grid, g_mu(mu_grid, full_hat$xi80_hat, full_hat$th80_hat),
        lwd = 2, col = "blue")

  legend("topright", legend = c("50m", "80m"), col = c("red", "blue"), lwd = 2, bty = "n")

} else {
  mu_grid <- seq(red_hat$xi_hat - 6 * red_hat$th_hat, red_hat$xi_hat + 10 * red_hat$th_hat, length.out =

  plot(mu_grid, g_mu(mu_grid, red_hat$xi_hat, red_hat$th_hat),
       type = "l", lwd = 2,
       xlab = expression(mu), ylab = "density",
       main = "Extreme Value mixing density g(mu | xi, theta) (reduced model)")
}


# ============================================================
# Q2: E(mu) = xi + gamma*theta  (delta method)
# ============================================================

wald_ci_mean_mu <- function(fit) {
  # input:  fit with (xi_hat, theta_hat, fInf) on (xi, logtheta) scale
  # output: eta_hat, SE, Wald CI for E(mu)
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V <- fit$fInf

  eta_hat <- xi_hat + gamma * th_hat

  # gradient wrt (xi, logtheta):
  # d/dxi = 1
  # d/dlogtheta = gamma * theta
  grad_vec <- c(1, gamma * th_hat)

  se <- sqrt(drop(t(grad_vec) %*% V %*% grad_vec))
  ci <- eta_hat + c(-1, 1) * qnorm(0.975) * se

  list(eta_hat = eta_hat, se = se, ci95 = ci)
}

ci50_eta <- wald_ci_mean_mu(fit50)
ci80_eta <- wald_ci_mean_mu(fit80)


# ============================================================
# Q3: Median of Y under mixture (root solve + delta method)
```

```r
# ============================================================

FY_stable <- function(y, xi, theta, zL = -100, zU = 100) {
  # input:  y, xi, theta
  # output: F_Y(y | xi, theta) under marginal mixture
  integrand <- function(z) {
    pnorm(y - (xi + theta * z)) * exp(-z - exp(-z))
  }

  val <- integrate(integrand, lower = zL, upper = zU,
                   rel.tol = 1e-8, abs.tol = 0, subdivisions = 5000)$value

  pmin(pmax(val, 0), 1)
}

median_y <- function(xi, theta, zL = -100, zU = 100) {
  # input:  xi, theta
  # output: median m solving F_Y(m | xi,theta) = 0.5
  f <- function(y) FY_stable(y, xi, theta, zL, zU) - 0.5

  y0 <- xi + gamma * theta
  lo <- y0 - 10 * theta - 5
  hi <- y0 + 10 * theta + 5

  for (k in 1:60) if (f(lo) > 0) lo <- lo - (2 + theta) else break
  for (k in 1:60) if (f(hi) < 0) hi <- hi + (2 + theta) else break

  uniroot(f, interval = c(lo, hi))$root
}

wald_ci_median_y <- function(fit, zL = -100, zU = 100) {
  # input:  fit on (xi, logtheta) scale
  # output: median plug-in + delta-method Wald CI
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V <- fit$fInf

  h <- function(par) {
    xi <- par[1]
    th <- exp(par[2])
    median_y(xi, th, zL = zL, zU = zU)
  }

  par_hat <- c(xi_hat, log(th_hat))
  m_hat <- h(par_hat)

  g <- grad(h, par_hat)
  se <- sqrt(drop(t(g) %*% V %*% g))
  ci <- m_hat + c(-1, 1) * qnorm(0.975) * se

  list(median_hat = m_hat, se = se, ci95 = ci)
}
```

```r
ci50_med <- wald_ci_median_y(fit50, zL = -100, zU = 100)
ci80_med <- wald_ci_median_y(fit80, zL = -100, zU = 100)


# ============================================================
# Q4: P(4 <= mu <= 17) under EV mixing (delta method)
# ============================================================

G_mu <- function(u, xi, theta) {
  # input:  u, xi, theta
  # output: EV CDF G(u | xi,theta)
  exp(-exp(-(u - xi) / theta))
}

wald_ci_p_between_mu <- function(fit, a = 4, b = 17) {
  # input:  fit on (xi, logtheta) scale
  # output: plug-in probability + delta-method Wald CI
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V <- fit$fInf

  h <- function(par) {
    xi <- par[1]
    th <- exp(par[2])
    G_mu(b, xi, th) - G_mu(a, xi, th)
  }

  par_hat <- c(xi_hat, log(th_hat))
  p_hat <- h(par_hat)

  g <- grad(h, par_hat)
  se <- sqrt(drop(t(g) %*% V %*% g))

  ci <- p_hat + c(-1, 1) * qnorm(0.975) * se
  ci <- pmin(pmax(ci, 0), 1)

  list(p_hat = p_hat, se = se, ci95 = ci)
}

ci50_p <- wald_ci_p_between_mu(fit50, a = 4, b = 17)
ci80_p <- wald_ci_p_between_mu(fit80, a = 4, b = 17)


# ============================================================
# Q6: Monte Carlo approximation of E(mu) (conditional on MLEs)
# ============================================================

set.seed(43)

rEV <- function(n, xi, theta) {
  # input:  n, xi, theta
  # output: n draws from EV(xi,theta) via inverse-CDF
  u <- runif(n)
```

```r
  xi - theta * log(-log(u))
}

mc_E_mu <- function(fit, M = 20000) {
  # input:  fit with (xi_hat, theta_hat); number of draws M
  # output: MC mean + MC CI; also plug-in xi + gamma*theta for comparison
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat

  mu_draws <- rEV(M, xi_hat, th_hat)

  mc_mean <- mean(mu_draws)
  mc_se <- sd(mu_draws) / sqrt(M)
  mc_ci <- mc_mean + c(-1, 1) * qnorm(0.975) * mc_se

  plug_in <- xi_hat + gamma * th_hat

  list(M = M, plug_in = plug_in, mc_mean = mc_mean, mc_ci95 = mc_ci)
}

mc50 <- mc_E_mu(fit50, M = 20000)
mc80 <- mc_E_mu(fit80, M = 20000)
```